Worksheet — 2.1 Elements of Computational Thinking — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Task 1 — Key-term match 1 → C, 2 → E, 3 → D, 4 → A, 5 → B.

Task 2 — Inputs / outputs / preconditions - (a) Inputs (any three): student ID; chosen room (room number); chosen date/time slot; (optionally) duration / number of people. - (b) Outputs (any two): the list of available rooms displayed; a confirmation message / booking reference; or a rejection message if unavailable. - (c) Precondition (any one): the student ID must be valid/registered before a booking is accepted; **or** the chosen room and slot must exist; **or** the chosen slot must currently be free. (Note: validating the ID and checking availability are *processes*, not inputs/outputs.)

Task 3 — Abstraction (keep/remove) - KEEP: room number/ID, maximum capacity, which slots are already booked — these are needed to decide and store a booking. - REMOVE: carpet colour, make of chairs — irrelevant to whether a room can be booked. They are irrelevant *to this problem* because the booking decision depends only on identity, capacity and availability, never on décor.

Task 4 — Decomposition (model order) 1. Get/validate the student ID. 2. Display the list of available rooms. 3. Get the student's chosen room and time slot. 4. Check the chosen slot is still free (availability check). 5. Store the booking in the database. 6. Print/display the confirmation (or rejection). Dependency example: step 5 (store) depends on the output of step 4 (the availability check must pass first); step 4 depends on the chosen room/slot from step 3.

Task 5 — Decision points (examples) - DP1: `IF studentID is valid` → TRUE: continue to room selection; FALSE: show "unknown ID / please register" and stop. - DP2: `IF chosenSlot is free` (e.g. `IF slotStatus = "free"`) → TRUE: store the booking and confirm; FALSE: reject and ask the student to choose another slot. - (Other valid conditions: `IF seatsRequested <= roomCapacity`, `IF bookingTime >= openingTime AND bookingTime <= closingTime`.)

Task 6 — Concurrency - (a) Independent (no data dependency): e.g. loading the available-rooms list for the screen **while** the system logs the previous request; refreshing one room's availability while another student's confirmation email is sent; rendering the UI while a background database query runs. - (b) Sequential (must stay ordered): you must **check availability before storing the booking**, and **store the booking before printing the confirmation** — each needs the previous step's result. - (c) Benefit: faster/more responsive — the interface stays usable while a slow query runs, and multiple students can be served at once on multi-core hardware. Trade-off: harder to design and debug; risk of a **race condition** if two students book the same slot at the same instant (needs synchronisation/locking), and concurrency adds overhead so it is not automatically faster.

Task 7 — Abstraction vs decomposition 1. A 2. D 3. A 4. D 5. A 6. D

Task 8 — All five skills in SnackTrack (model answers)

Thinking skill	Model example (must be scenario-specific)
Abstractly	The menu shows only photo, name and price — the recipe, supplier and kitchen rota are hidden because they are irrelevant to choosing lunch; each item is modelled as just (name, price, stock level).
Ahead	The designers identified the **inputs** (ID card tap, item choices), **outputs** (receipt, on-screen menu) and **preconditions** (student has enough credit; item in stock) before coding; the menu is **cached** on the kiosk from 7 am so it is not re-fetched for every student.
Procedurally	"Serve a student" is decomposed into ordered sub-procedures: read card → display menu → take selection → check credit and stock → take payment → update stock → print receipt.
Logically	Decision points such as `IF credit >= totalCost` and `IF stockLevel > 0` determine whether the sale proceeds or an error message is shown.
Concurrently	While one student's payment is processed, a second thread refreshes the "sold out" labels on the other kiosks — two activities happening at the same time with no shared dependency on that student's transaction.

Task 9 — Precondition, caching, or neither? 1 = **Precondition** · 2 = **Caching** · 3 = **Precondition** · 4 = **Neither** (it is an output/step) · 5 = **Caching** · 6 = **Precondition** (binary search requires sorted data) · 7 = **Neither** (hardware detail, irrelevant to thinking ahead). - (b) Stale examples: row 2 — the kitchen runs out of an item or changes the menu mid-day; row 5 — a booking or cancellation changes which rooms are free while the old list is still in RAM. - (c) If the precondition is false the program should **not** continue as normal: e.g. row 1 — reject the transaction and display "card not recognised"; row 3 — abandon the write and tell the student the slot has just been taken; row 6 — sort the array first (or use a linear search).

Challenge box — caching Caching stores the "rooms free" list in fast memory so repeated requests are served without re-querying the database each time — this is faster, reduces database load/traffic and improves responsiveness when many students ask at once. Trade-off: it uses extra storage, and the cached list can become **stale** when a booking or cancellation changes availability but the cache still shows the old state — a student could be shown a room that has just been taken. Handle this by **invalidating/updating the cache** whenever a booking or cancellation occurs, or by setting a short expiry time so the list is refreshed regularly.

Section B — model answers

Q1 [2] (AO1) — Thinking ahead means planning, **before implementation**, what the program will need: identifying the **inputs and outputs** in advance (1); and identifying **preconditions** that must hold for it to work, plus deciding what can be prepared/stored ready for reuse, such as **caching** values or planning **reusable components** (1). *(Any two distinct strands, 1 mark each.)*

Q2 [2] (AO2) — Any two, 1 mark each: the **number plate** read by the entrance camera (or typed at the pay station); the **bay sensor signals** (occupied/free); the driver's **card payment details**. *(Do not accept outputs such as the bay number displayed or the barrier rising; do not accept processes such as "calculating the fee".)*

Q3 [3] (AO2) - Abstraction is **removing/hiding unnecessary detail** so that only the information relevant to the problem is modelled (1). - Here the model keeps only the **bay number and an occupied/free flag** and discards paint colour and floor surface (1). - This is appropriate because assigning a bay and counting free spaces depend **only on identity and occupancy** — the discarded details can never change the outcome, and the simpler model is faster to process and easier to store/maintain (1).

Q4 [4] (AO2) — One mark per sub-problem in a workable order, e.g.: 1. Capture the image and **read the number plate** (1). 2. **Check whether the car park is full** (compare free-space count with zero) (1). 3. **Find/assign a free bay** and record the plate + entry time against it (1). 4. **Display the bay number** on the screen and **raise the barrier** (1). *(Accept splitting differently, e.g. "record entry time in database" as a step; the order must respect dependencies — you cannot assign a bay before checking fullness.)*

Q5 [3] (AO2) — Example: - Condition: `IF freeSpaces > 0` (or `IF carParkFull = FALSE`) (1). - TRUE branch: assign a bay, display its number, raise the entrance barrier (1). - FALSE branch: display "car park full", keep the barrier down (1). *(Other creditworthy conditions: `IF paymentAuthorised = TRUE` at exit → raise exit barrier / else request another card; `IF plateRecognised = TRUE` → look up entry time / else ask driver to enter plate manually. Condition must be Boolean and scenario-specific; both branches required.)*

Q6 [3] (AO2) - Benefit (up to 2): reading one cached total is far **quicker/cheaper** than polling hundreds of bay sensors for every sign update (1), so the signs update promptly and the sensor network/database is not overloaded by the many signs around town (1). - Drawback (1): the cached count can be **stale/out of date** — cars enter or leave during the 60 seconds between refreshes, so a sign may say spaces are free when the car park has just filled, sending drivers to a full car park.

Q7 [4] (AO2) - Thinking concurrently = recognising which parts of the problem can happen **at the same time** and designing the program so independent tasks run simultaneously (1). - Valid concurrent pair (1): e.g. processing a car at the **entrance camera** while a different driver pays at the **exit pay station** — they involve different cars and different data, so there is **no data dependency** (1 for justifying independence). - Problem (1): a **race condition** on shared data — e.g. two entrances (or an entry and an exit) updating the free-space count at the same instant could corrupt the count, or two cars could be assigned the **same bay**; this must be prevented with locking/synchronisation, which adds complexity and makes bugs harder to reproduce and debug.

Q8* [9] (AO3) — levels of response

Level	Marks	Descriptor (OCR style)
Level 3	7–9	A **thorough** discussion showing detailed understanding of concurrency. Both benefits and drawbacks are considered and are **consistently applied to ParkSmart** (entrances, exits, sensors, shared free-space count), not generic. There is a **well-developed line of reasoning which is clear and logically structured**; evaluative comments are **substantiated**, and a **justified recommendation** is made.
Level 2	4–6	A **reasonable** discussion of benefits and drawbacks, with **some application** to the scenario; some points generic or undeveloped. A line of reasoning with some structure; a recommendation may be given but is only partially justified.
Level 1	1–3	A **basic** answer: one or two relevant points, largely definitions with little or no application to ParkSmart; little or no evaluation; poorly structured.
—	0	No response or no response worthy of credit.

Indicative content — *benefits of concurrent design:* entrance, exits and sensor updates are largely independent, so they can genuinely overlap; better **responsiveness** (a queue at the entrance is not blocked while someone pays at the exit); scales across multiple entrances/pay stations and exploits multi-core hardware; a slow step (card authorisation) doesn't freeze the whole system. *Drawbacks:* shared data — the free-space count and the bay table are written by several processes, risking **race conditions** (two cars assigned one bay; corrupted count); needs locks/synchronisation which adds complexity, potential **deadlock**, and harder testing/debugging (bugs are timing-dependent and hard to reproduce); overhead of

coordination may outweigh gains if traffic is light. *Recommendation:* concurrent design with synchronised access to shared data is the defensible choice for a busy multi-entrance car park; a purely sequential design is only adequate for a very small site.

Model top-band answer (full prose):

> A concurrent design suits ParkSmart because its workload is naturally simultaneous: a car arriving at the entrance, a driver paying at the exit and hundreds of bay sensors reporting are independent events involving different cars and different data. Handling them concurrently means the entrance barrier does not queue behind a slow card authorisation at the pay station, so drivers are served faster at busy times; and with several entrances and pay stations, a sequential design would create exactly the kind of blocking delays that cause queues onto the road. Concurrency also exploits modern multi-core hardware and lets the town-centre signs be refreshed in the background without pausing entries.
>
> However, the processes are not fully independent: they all share the free-space count and the table of bays. If an entry process and an exit process update the count at the same instant, a race condition could corrupt it — or two cars arriving at two entrances could be assigned the same bay. Preventing this requires locks or atomic updates on the shared data, which adds design complexity, creates the possibility of deadlock, and makes testing harder because timing-dependent bugs are difficult to reproduce. A sequential design avoids all of this and would be simpler to build and verify.
>
> On balance, the developers should choose the **concurrent design, with synchronised (locked) access to the shared free-space count and bay table**. The shared data is small and updates are brief, so the locking cost is negligible, while the responsiveness benefit applies to every car at peak time; a sequential system would only be justified for a tiny single-entrance car park, which ParkSmart — a multi-storey site with several pay stations and town-wide signage — clearly is not. The concurrency risks are real but well-understood and cheap to control, whereas the queues caused by a sequential design cannot be engineered away.

(Section B total: 2 + 2 + 3 + 4 + 3 + 3 + 4 + 9 = 30 marks.)